

Distributed Transactions in Distributed Systems

1. Introduction to Distributed Transactions

A **distributed transaction** is a database transaction that spans multiple networked databases or systems, ensuring that operations occurring on different nodes remain consistent and reliable. These transactions typically involve multiple participants across different systems, making coordination and consistency crucial.

Why Are Distributed Transactions Important?

- In distributed systems, multiple databases or services may need to update their records as part of a single logical operation.
- Failures in one part of the transaction should not leave the system in an inconsistent state.
- Transactions should remain **atomic**, **consistent**, **isolated**, and **durable** (ACID properties).

Examples of Distributed Transactions

1. **Bank Transfers:** A money transfer from Bank A to Bank B must withdraw money from one account and deposit it into another in a consistent manner.
2. **E-commerce Orders:** Placing an order involves updating inventory, processing payment, and confirming shipment, all across different services.
3. **Microservices Communication:** A user registration process might involve inserting user data into an authentication database and sending a welcome email, requiring coordination between different services.

2. Properties of Transactions (ACID Properties)

Transactions in distributed systems must adhere to the **ACID properties**, which ensure data integrity and reliability.

a) Atomicity (A) - “All or Nothing” Principle

- Ensures that a transaction is treated as a **single unit of work**. Either the entire transaction executes successfully, or none of it does.
- If a failure occurs in any step, the entire transaction must be **rolled back** to its initial state.
- **Example:** In a bank transfer, if the system fails after deducting money from one account but before depositing it in the recipient’s account, the deducted amount must be restored.

b) Consistency (C) - Maintaining Data Integrity

- Guarantees that a transaction brings the database from one **valid state** to another.
- Transactions must **respect all integrity constraints** such as referential integrity (foreign keys), unique constraints, and business rules.
- **Example:** If an e-commerce system allows a customer to order an item, it must ensure the item exists in stock before confirming the order.

c) Isolation (I) - Preventing Concurrent Issues

- Ensures that transactions execute **independently** of each other, preventing issues like:
 - **Dirty Reads:** Reading uncommitted changes from another transaction.
 - **Non-Repeatable Reads:** Data changes between reads in the same transaction.
 - **Phantom Reads:** New records appear in queries during the same transaction.
- **Example:** In an online ticket booking system, two users should not be able to book the same seat simultaneously.

d) Durability (D) - Ensuring Data Persistence

- Once a transaction is **committed**, its changes must be permanently stored, even in the event of a system failure.
- Databases achieve durability through:
 - **Write-ahead logging (WAL):** Changes are first logged before committing.
 - **Replication:** Data is stored in multiple locations for fault tolerance.

- **Example:** If an online order is placed and the system crashes, the order should still be available when the system restarts.

3. Primitives of Distributed Transactions

Since distributed transactions involve multiple independent systems, **coordinating them efficiently** is essential. Various primitives and protocols help in maintaining ACID properties.

a) Two-Phase Commit (2PC) Protocol

- A widely used protocol in distributed databases to ensure **atomicity**.
- **Phases of 2PC:**
 1. **Prepare Phase:**
 - The transaction coordinator sends a **prepare request** to all participants.
 - Each participant checks if they can commit the transaction (resources available, constraints met, etc.).
 - If ready, they respond **YES**, otherwise **NO**.
 2. **Commit Phase:**
 - If all participants responded **YES**, the coordinator sends a **commit request**.
 - If any participant responded **NO**, the coordinator sends a **rollback request** to ensure consistency.
- **Drawback of 2PC:**
 - If the coordinator crashes before sending the final commit/rollback, participants may remain in an uncertain state.

b) Three-Phase Commit (3PC) Protocol

- Addresses the limitations of **2PC** by adding a **pre-commit phase**, reducing blocking issues.
- **Phases of 3PC:**

1. **Prepare Phase:** Similar to 2PC, the coordinator asks participants if they are ready.
 2. **Pre-Commit Phase:** If all say **YES**, the coordinator sends a **pre-commit request**, ensuring all participants prepare for commitment.
 3. **Commit Phase:** After a timeout, if no failures occur, the coordinator instructs participants to commit.
- **Advantage:** Reduces the risk of blocking in case of coordinator failure.

c) Concurrency Control Mechanisms

- Prevents conflicts when multiple transactions access shared data.
- Common techniques include:
 - **Two-Phase Locking (2PL):** Locks resources in two phases (acquire locks, then release).
 - **Timestamp Ordering:** Ensures transactions execute in the order of their timestamps.
 - **Optimistic Concurrency Control:** Assumes no conflict will occur and validates at commit time.

d) Distributed Deadlock Detection and Handling

- Deadlocks occur when two or more transactions wait for each other indefinitely.
- **Solutions include:**
 - **Wait-for Graphs:** Detects circular waits and aborts one transaction.
 - **Timeout Mechanism:** Transactions abort automatically after a time limit.

e) Data Replication and Partitioning

- **Replication:** Storing copies of data across multiple nodes ensures fault tolerance.
- **Partitioning:** Dividing large datasets into smaller, manageable chunks improves performance.
- **Example:** Google's Bigtable and Amazon DynamoDB use replication and partitioning for scalability.

4. Challenges of Distributed Transactions

Despite the benefits, distributed transactions present several challenges:

a) Network Latency

- Communication between distributed nodes introduces delays, affecting transaction speed.

b) Partial Failures

- A failure in one node can leave the system in an inconsistent state.

c) Scalability Issues

- Coordinating transactions across many nodes can become complex as the system scales.

d) CAP Theorem Limitations

- The **CAP theorem** states that a distributed system can only provide two of the following three properties at a time:
 - **Consistency**
 - **Availability**
 - **Partition Tolerance**
- This means strict ACID compliance may impact system availability.

5. Conclusion

Distributed transactions ensure reliable execution of operations across multiple systems. **ACID properties** maintain data integrity, while **primitives like 2PC, 3PC, and concurrency control mechanisms** help in coordinating transactions across distributed environments. However, challenges such as **network latency, partial failures, and scalability** must be addressed to ensure optimal performance.

ACID Properties in DBMS

